Reviewer Pack — Resolution Width Lower Bound via Graph Betti Numbers in Tsei…

Entry 802deca2e3a4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 12:29:34 UTC

Resolution Width Lower Bound via Graph Betti Numbers in Tseitin Formulas

Entry ID: `802deca2e3a4` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-19 12:29:34 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology **Field B** (complexity object): Resolution Width of Tseitin Formulas

Statement:

> For a Tseitin formula derived from a connected graph G with m edges and n vertices, the resolution width is at least (m - n + 1). This bound holds for all graphs where the Tseitin formula is unsatisfiable and the graph has no multiple edges or loops.

Rationale (proposer's reasoning):

> Betti numbers capture the cyclomatic complexity of graphs, which directly relates to the number of independent cycles in the constraint graph. These cycles create structural dependencies that may necessitate wider resolution proofs, as each cycle could require additional clauses to resolve.

Taxonomy category: `PROOF_COMPLEXITY_TSEITIN` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `a281ca1a53f4a14e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        max_row = i + max(range(i, n), key=lambda k: abs(A[k][i]))
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        factor = Fraction(A[i][i])
        for j in range(n):
            A[i][j] /= factor
        b[i] /= factor
        for k in range(n):
            if k != i:
                factor = Fraction(A[k][i])
                for j in range(n):
                    A[k][j] -= factor * A[i][j]
                b[k] -= factor * b[i]

def matrix_multiply(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = Fraction(0)
    sign = 1
    for j in range(n):
        submatrix = [[A[i][k] for k in range(n) if k != j] for i in range(1, n)]
        det += sign * A[0][j] * determinant(submatrix)
```

```
        sign *= -1
    return det

def is_connected(graph):
    n = len(graph)
    visited = [False] * n
    stack = [0]
    while stack:
        node = stack.pop()
        if not visited[node]:
            visited[node] = True
            for neighbor in range(n):
                if graph[node][neighbor] and not visited[neighbor]:
                    stack.append(neighbor)
    return all(visited)

def betti_number(graph):
    n = len(graph)
    A = [[0] * n for _ in range(n)]
    b = [0] * n
    for i in range(n):
        for j in range(i + 1, n):
            if graph[i][j]:
                A[i][j] = A[j][i] = -1
                b[i] += 1
                b[j] += 1
    gaussian_elimination(A, b)
    rank = sum(1 for row in A if any(row))
    return rank

def resolution_width(graph):
    n = len(graph)
    literals = list(range(n))
    clauses = []
    for i in range(n):
        for j in range(i + 1, n):
            if graph[i][j]:
                clauses.append([literals[0]] + [-x for x in literals[1:]])
    A = [[0] * len(clauses) for _ in range(len(literals))]
    b = [0] * len(clauses)
    for i, clause in enumerate(clauses):
        for j in range(len(literals)):
            if any(x == literals[j] or x == -literals[j] for x in clause):
                A[j][i] += 1
                b[i] += 1
    gaussian_elimination(A, b)
    rank = sum(1 for row in A if any(row))
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    graph = [[0] * n for _ in range(n)]
    for _ in range(random.randint(1, n - 1)):
        u, v = random.sample(range(n), 2)
        if not graph[u][v]:
```

```
            graph[u][v] = graph[v][u] = 1
    if not is_connected(graph):
        return {
            "metric_name": "resolution_width",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "graph_not_connected"
        }
    betti = betti_number(graph)
    width = resolution_width(graph)
    return {
        "metric_name": "resolution_width",
        "metric_value": width,
        "instances_tested": 1,
        "conjecture_holds": width >= betti,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 2
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)
    metric_values = [r["metric_value"] for r in results if r["metric_value"] is not None]
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)
    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={sum(metric_values)/len(metric_values)} std=0.0 support_fra
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={sum(metric_values)/len(metric_values)} std={math.sqrt(sum(
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"resolution_width < betti_number\" first_failing
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
stances_tested": 1, "conjecture_holds": False, "counterexample": "graph_not_connected"}
TRIAL: {"seed": 593, "metric_name": "resolution_width", "metric_value": None, "instances_tested": 1,
TRIAL: {"seed": 631, "metric_name": "resolution_width", "metric_value": None, "instances_tested": 1,
TRIAL: {"seed": 677, "metric_name": "resolution_width", "metric_value": None, "instances_tested": 1,
TRIAL: {"seed": 727, "metric_name": "resolution_width", "metric_value": None, "instances_tested": 1,
TRIAL: {"seed": 773, "metric_name": "resolution_width", "metric_value": None, "instances_tested": 1,
TRIAL: {"seed": 821, "metric_name": "resolution_width", "metric_value": None, "instances_tested": 1,
TRIAL: {"seed": 877, "metric_name": "resolution_width", "metric_value": None, "instances_tested": 1,
TRIAL: {"seed": 929, "metric_name": "resolution_width", "metric_value": None, "instances_tested": 1,

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b6c98454.py", line 143, in <module>
```

```
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b6c98454.py", line 143, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                              ~^^^^^^^^
KeyError: 'seed'
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed during execution, preventing reliable evaluation of conjecture validity. | next: Implement robustness checks for graph connectivity in test setup

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	84322
2	preregistration	ollama_remote	qwen3:8b	0	0	27446
3	novelty	ollama_remote	qwen3:8b	0	0	24156
4	novelty	ollama_remote	qwen3:8b	0	0	15661
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11657
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10286
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13709
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15690
9	judge	ollama_remote	qwen3:8b	0	0	19428

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 222356 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/802deca2e3a4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/802deca2e3a4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/802deca2e3a4.tar.gz` (if generated)